

• • • • • • • • • •

# COMPREHENSIVE GUIDE TO Nested If-Else Statements in Python

---

Complete Reference · All Concepts · Rich Examples

```
if condition1:  
    if condition2:  
        ... # deeper nesting  
    elif condition3:  
        ...  
    else:  
        ...
```

July 2, 2026

## Contents

---

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Conditional Statements</b>                                       | <b>3</b>  |
| 1.1      | What is a Conditional Statement? . . . . .                                          | 3         |
| 1.2      | Python's Decision-Making Keywords . . . . .                                         | 3         |
| <b>2</b> | <b>The Basic if Statement</b>                                                       | <b>3</b>  |
| 2.1      | Syntax . . . . .                                                                    | 3         |
| 2.2      | Flowchart — Simple if . . . . .                                                     | 4         |
| 2.3      | Example 1 — Simple if . . . . .                                                     | 4         |
| <b>3</b> | <b>The if-else Statement</b>                                                        | <b>4</b>  |
| 3.1      | Syntax . . . . .                                                                    | 4         |
| 3.2      | Flowchart — if-else . . . . .                                                       | 5         |
| 3.3      | Example 2 — Even or Odd . . . . .                                                   | 5         |
| 3.4      | Example 3 — Pass or Fail . . . . .                                                  | 5         |
| <b>4</b> | <b>The if-elif-else Ladder</b>                                                      | <b>5</b>  |
| 4.1      | Syntax . . . . .                                                                    | 5         |
| 4.2      | Flowchart — if-elif-else Ladder . . . . .                                           | 6         |
| 4.3      | Example 4 — Grade Classification . . . . .                                          | 6         |
| 4.4      | Example 5 — BMI Calculator . . . . .                                                | 6         |
| <b>5</b> | <b>Nested If-Else — The Core Topic</b>                                              | <b>7</b>  |
| 5.1      | What is Nesting? . . . . .                                                          | 7         |
| 5.2      | Syntax — Two Levels Deep . . . . .                                                  | 7         |
| 5.3      | Syntax — Three Levels Deep . . . . .                                                | 7         |
| 5.4      | How Python Matches <code>else</code> to <code>if</code> . . . . .                   | 8         |
| 5.5      | Flowchart — Nested if-else (Two Levels) . . . . .                                   | 8         |
| 5.6      | Example 6 — Login System . . . . .                                                  | 8         |
| 5.7      | Example 7 — Number Classification . . . . .                                         | 9         |
| 5.8      | Example 8 — Admission Eligibility . . . . .                                         | 9         |
| <b>6</b> | <b>All Possible Uses and Patterns</b>                                               | <b>10</b> |
| 6.1      | Pattern 1 — Standalone if (No else) . . . . .                                       | 10        |
| 6.2      | Pattern 2 — Multiple Independent ifs . . . . .                                      | 10        |
| 6.3      | Pattern 3 — Nested if Inside elif . . . . .                                         | 11        |
| 6.4      | Pattern 4 — Nested if Inside else . . . . .                                         | 11        |
| 6.5      | Pattern 5 — Deep Nesting (4+ Levels) . . . . .                                      | 11        |
| 6.6      | Pattern 6 — Logical Operators Inside if . . . . .                                   | 12        |
| 6.7      | Pattern 7 — Comparison Operators Reference . . . . .                                | 13        |
| 6.8      | Pattern 8 — Chained Comparisons . . . . .                                           | 13        |
| 6.9      | Pattern 9 — Membership Tests with <code>in</code> and <code>not in</code> . . . . . | 13        |
| 6.10     | Pattern 10 — Identity Tests with <code>is</code> and <code>is not</code> . . . . .  | 14        |
| 6.11     | Pattern 11 — Truthy and Falsy Values . . . . .                                      | 14        |
| 6.12     | Pattern 12 — Ternary (Conditional Expression) . . . . .                             | 15        |
| 6.13     | Pattern 13 — if Calling Functions . . . . .                                         | 15        |
| 6.14     | Pattern 14 — if Inside Loops (FizzBuzz) . . . . .                                   | 16        |
| 6.15     | Pattern 15 — if with User Input . . . . .                                           | 16        |
| 6.16     | Pattern 16 — if in List / Dict Comprehensions . . . . .                             | 17        |

|           |                                                 |           |
|-----------|-------------------------------------------------|-----------|
| <b>7</b>  | <b>Real-World Application Examples</b>          | <b>17</b> |
| 7.1       | ATM Simulation . . . . .                        | 17        |
| 7.2       | Traffic Light System . . . . .                  | 18        |
| 7.3       | E-Commerce Discount Engine . . . . .            | 18        |
| 7.4       | Weather Advisory System . . . . .               | 19        |
| <b>8</b>  | <b>Nesting Depth — Visual Decision Tree</b>     | <b>19</b> |
| <b>9</b>  | <b>The pass Statement in Conditionals</b>       | <b>20</b> |
| <b>10</b> | <b>Guard Clauses — Refactoring Deep Nesting</b> | <b>20</b> |
| <b>11</b> | <b>Common Mistakes and How to Avoid Them</b>    | <b>22</b> |
| <b>12</b> | <b>All Variants — Summary Overview</b>          | <b>23</b> |
| <b>13</b> | <b>Best Practices Summary</b>                   | <b>23</b> |
| <b>14</b> | <b>Conclusion</b>                               | <b>23</b> |

## 1 Introduction to Conditional Statements

### 1.1 What is a Conditional Statement?

In programming, a **conditional statement** (also called a *decision-making statement*) allows a program to **make decisions** and execute different blocks of code based on whether a condition is **True** or **False**. Python uses Boolean logic to evaluate conditions.

**Core Idea:** A conditional statement asks a yes/no question (*Is the condition True?*). Depending on the answer, a different code path is taken. Without conditionals, programs would execute every line in the same fixed order every time — they could not adapt to different inputs.

### 1.2 Python's Decision-Making Keywords

| Keyword           | Purpose                                                                        |
|-------------------|--------------------------------------------------------------------------------|
| <code>if</code>   | Starts a conditional block; executes if condition is <b>True</b>               |
| <code>elif</code> | Short for “else if”; checks another condition if the previous was <b>False</b> |
| <code>else</code> | Default fallback; executes when all above conditions are <b>False</b>          |
| <code>and</code>  | Logical AND — both sides must be <b>True</b>                                   |
| <code>or</code>   | Logical OR — at least one side must be <b>True</b>                             |
| <code>not</code>  | Logical NOT — inverts a Boolean value                                          |
| <code>in</code>   | Membership test — checks if a value exists in a sequence                       |
| <code>is</code>   | Identity test — checks if two variables point to the same object               |
| <code>pass</code> | Null operation — empty placeholder inside a block                              |

## 2 The Basic if Statement

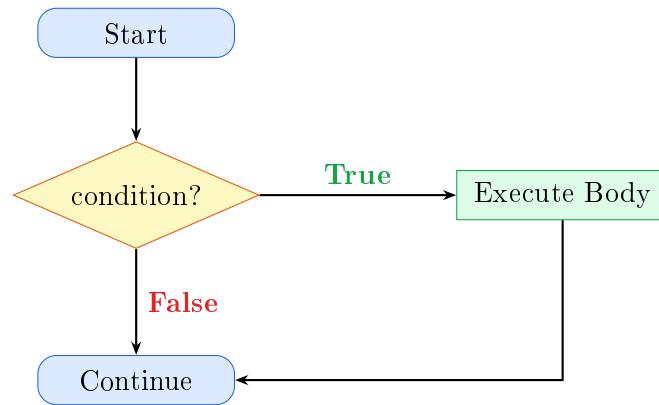
### 2.1 Syntax

```
if condition:
    statement_1      # executed only when condition is True
    statement_2
    ...
```

#### Important Rules

- The `if` keyword is followed by a **condition** and a **colon (:)**.
- The body must be **indented** — 4 spaces per level (PEP 8 standard).
- Python uses **indentation** instead of curly braces to define blocks.
- If the condition evaluates to **False**, the body is **skipped entirely**.

## 2.2 Flowchart — Simple if



## 2.3 Example 1 — Simple if

```
1 age = 20
2
3 if age >= 18:
4     print("You are an adult.")      # runs because 20 >= 18 is True
5     print("You can vote.")
6
7 print("Program ends.")              # always runs regardless
```

Listing 1: Basic if statement

```
You are an adult.
You can vote.
Program ends.
```

# 3 The if-else Statement

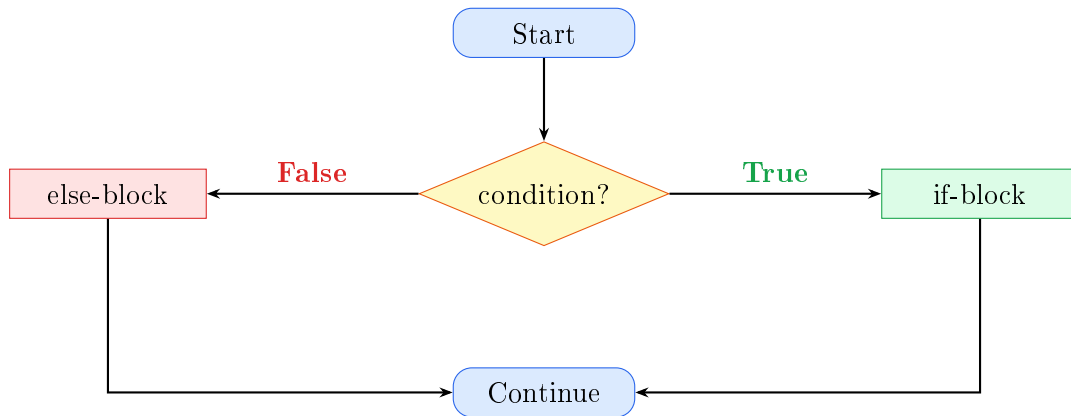
## 3.1 Syntax

```
if condition:
    true_block      # runs when condition is True
else:
    false_block     # runs when condition is False
```

### Key Rule

**else never** has a condition. It is the *catch-all* block — it runs when the **if** condition is **False**. **else** must always appear **last** and only once per **if** block.

### 3.2 Flowchart — if-else



### 3.3 Example 2 — Even or Odd

```
1 number = 7
2
3 if number % 2 == 0:
4     print(f"{number} is Even")
5 else:
6     print(f"{number} is Odd")    # 7 % 2 == 1, so this runs
```

Listing 2: if-else: even or odd check

```
7 is Odd
```

### 3.4 Example 3 — Pass or Fail

```
1 marks = 45
2
3 if marks >= 50:
4     print("Result: PASS")
5 else:
6     print("Result: FAIL")    # 45 < 50, so this runs
```

Listing 3: if-else: pass/fail decision

```
Result: FAIL
```

## 4 The if-elif-else Ladder

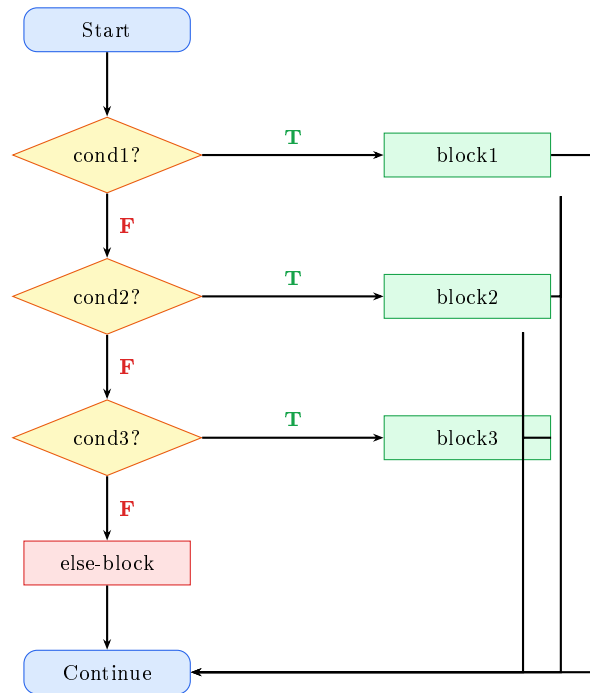
### 4.1 Syntax

```
if condition1:
    block1
elif condition2:
    block2
elif condition3:
    block3
...
else:
    default_block
```

### How the Ladder Works

Python checks conditions **top to bottom**. As soon as one condition is **True**, its block executes and **all remaining elif/else** blocks are **skipped entirely**. Only **one** block ever runs.

## 4.2 Flowchart — if-elif-else Ladder



## 4.3 Example 4 — Grade Classification

```

1 marks = 78
2
3 if marks >= 90:
4     grade = "A+"
5 elif marks >= 80:
6     grade = "A"
7 elif marks >= 70:
8     grade = "B"           # 78 falls here -- this block runs
9 elif marks >= 60:
10    grade = "C"
11 elif marks >= 50:
12    grade = "D"
13 else:
14    grade = "F"           # only reached if marks < 50
15
16 print(f"Marks: {marks} => Grade: {grade}")

```

Listing 4: if-elif-else: grade system

```
Marks: 78 => Grade: B
```

## 4.4 Example 5 — BMI Calculator

```

1 weight = 70    # kg
2 height = 1.75  # metres

```

```
3
4 bmi = weight / (height ** 2)      # 22.86
5
6 if bmi < 18.5:
7     category = "Underweight"
8 elif bmi < 25.0:
9     category = "Normal weight"    # 22.86 falls here
10 elif bmi < 30.0:
11     category = "Overweight"
12 else:
13     category = "Obese"
14
15 print(f"BMI: {bmi:.2f} => {category}")
```

Listing 5: if-elif-else: BMI category

```
BMI: 22.86 => Normal weight
```

## 5 Nested If-Else — The Core Topic

### 5.1 What is Nesting?

**Nesting** means placing one `if/elif/else` structure **inside** the body of another. This allows programs to test *multiple layers of conditions*, where each inner condition is only evaluated after its outer condition passes.

**Golden Rule of Nesting:** Each level of nesting must be indented **consistently and further** than the level above. Python uses indentation to determine which `else/elif` belongs to which `if`.

### 5.2 Syntax — Two Levels Deep

```
if outer_condition:
    # outer-if body
    if inner_condition:
        inner_true_block
    else:
        inner_false_block
else:
    outer_false_block
```

### 5.3 Syntax — Three Levels Deep

```
if condition_A:
    if condition_B:
        if condition_C:
            block_C_true
        else:
            block_C_false
    else:
        block_B_false
else:
    block_A_false
```



## 5.4 How Python Matches else to if

### Indentation is Everything

- Every `else/elif` belongs to the **nearest preceding if** at the **same indentation level**.
- Wrong indentation changes which `if` the `else` is attached to — silently changing program logic.
- Always use **4 spaces** per level (PEP 8). Never mix tabs and spaces.

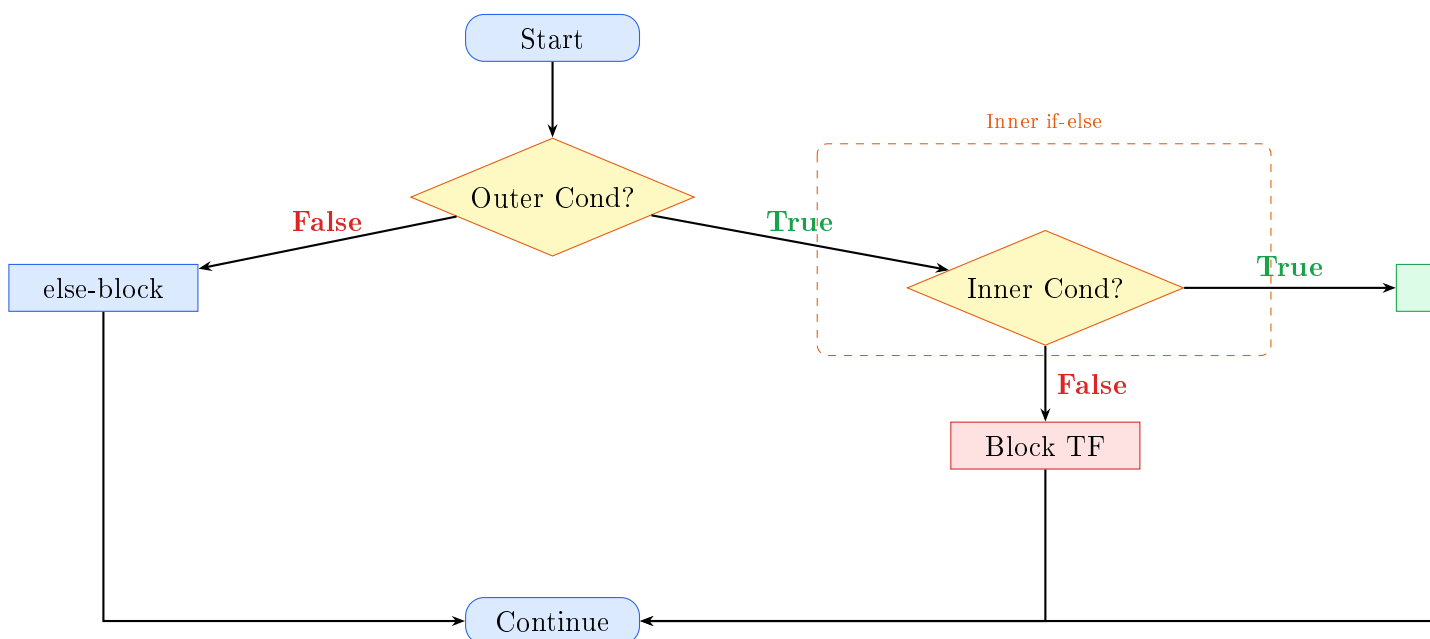
```

1 x = 10
2
3 # CORRECT -- else belongs to inner if
4 if x > 5:
5     if x > 8:
6         print("Greater than 8")
7     else:
8         print("Between 5 and 8")    # inner else
9
10 # INCORRECT (logic bug) -- wrong indentation
11 if x > 5:
12     if x > 8:
13         print("Greater than 8")
14 else:
15     print("5 or less")              # now attached to OUTER if!
                                     # this changes the meaning!

```

Listing 6: Correct vs incorrect else pairing

## 5.5 Flowchart — Nested if-else (Two Levels)



## 5.6 Example 6 — Login System

```

1 username = "admin"
2 password = "secure123"
3

```

```
4 input_user = "admin"
5 input_pass = "secure123"
6
7 if input_user == username:           # outer: check username
8     if input_pass == password:       # inner: check password
9         print("Login Successful!")
10        print("Welcome, admin!")
11    else:
12        print("Wrong password!")
13 else:
14    print("Username not found!")
```

Listing 7: Nested if-else: username + password check

```
Login Successful!
Welcome, admin!
```

### 5.7 Example 7 — Number Classification

```
1 num = -7
2
3 if num != 0:
4     if num > 0:
5         if num % 2 == 0:
6             print(f"{num} is Positive and Even")
7         else:
8             print(f"{num} is Positive and Odd")
9     else:           # num < 0
10        if num % 2 == 0:
11            print(f"{num} is Negative and Even")
12        else:
13            print(f"{num} is Negative and Odd") # -7 lands here
14 else:
15    print("The number is Zero")
```

Listing 8: Nested if-else: classify a number fully

```
-7 is Negative and Odd
```

### 5.8 Example 8 — Admission Eligibility

```
1 age      = 17
2 gpa      = 3.8
3 has_docs = True
4
5 if age >= 16:           # Level 1
6     if gpa >= 3.5:      # Level 2
7         if has_docs:    # Level 3
8             print("Admission APPROVED")
9         else:
10            print("Submit required documents")
11    elif gpa >= 2.5:
12        print("Eligible for conditional admission")
13    else:
```

```
14         print("GPA too low")
15     else:
16         print("Minimum age is 16")
```

Listing 9: Three-level nested if-else: admission check

```
Admission APPROVED
```

## 6 All Possible Uses and Patterns

### 6.1 Pattern 1 — Standalone if (No else)

Useful when you only want to **do something extra** when a condition is true, and do nothing otherwise.

```
1 speed = 95      # km/h
2 limit = 80
3
4 fine = 0
5 if speed > limit:      # no else needed
6     fine = (speed - limit) * 5
7     print(f"Speeding fine: ${fine}")
8
9 print("Drive safely!")
```

Listing 10: Standalone if — speeding fine

```
Speeding fine: $75
Drive safely!
```

### 6.2 Pattern 2 — Multiple Independent ifs

Each **if** is **independent** — all conditions are checked regardless of each other.

```
1 temperature = 35
2
3 if temperature > 30:
4     print("It's hot outside.")      # runs
5 if temperature > 40:
6     print("Extreme heat warning!")  # does NOT run (35 < 40)
7 if temperature > 20:
8     print("Stay hydrated.")         # runs
```

Listing 11: Multiple independent if statements

```
It's hot outside.
Stay hydrated.
```

#### Independent ifs vs elif Chain

Use **elif** when conditions are **mutually exclusive** (only one should match). Use separate **ifs** when **multiple conditions** can be true simultaneously and all should trigger independently.

### 6.3 Pattern 3 — Nested if Inside elif

```
1 score      = 72
2 attendance = 85
3
4 if score >= 90:
5     print("Distinction")
6 elif score >= 70:           # score 72 enters here
7     if attendance >= 80:    # check sub-condition
8         print("First Class") # attendance 85 >= 80: runs
9     else:
10        print("Pass (low attendance)")
11 elif score >= 50:
12     print("Pass")
13 else:
14     print("Fail")
```

Listing 12: Nested if inside elif block

```
First Class
```

### 6.4 Pattern 4 — Nested if Inside else

```
1 day = "Saturday"
2
3 if day in ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"):
4     print("Weekday -- go to work!")
5 else:
6     if day == "Saturday":
7         print("Weekend -- rest day!")
8     elif day == "Sunday":
9         print("Weekend -- family day!")
10    else:
11        print("Invalid day.")
```

Listing 13: Nested if inside the else block

```
Weekend -- rest day!
```

### 6.5 Pattern 5 — Deep Nesting (4+ Levels)

```
1 account_active = True
2 has_balance    = True
3 amount         = 500
4 daily_limit    = 1000
5
6 if account_active:
7     if has_balance:
8         if amount > 0:
9             if amount <= daily_limit:
10                print(f"Transferring ${amount}...")
11                print("Transaction Successful!")
12            else:
13                print("Exceeds daily transfer limit!")
14        else:
```

```
15         print("Invalid amount.")
16     else:
17         print("Insufficient balance.")
18 else:
19     print("Account is not active.")
```

Listing 14: Deep nesting: bank transaction system

```
Transferring $500...
Transaction Successful!
```

### Best Practice — Avoid Deep Nesting

Nesting beyond **3–4 levels** makes code hard to read and maintain. Consider using **guard clauses** (early returns), **logical operators** (**and/or**), or breaking logic into separate functions.

## 6.6 Pattern 6 — Logical Operators Inside if

Logical operators let you **combine** multiple conditions on a single line.

```
1  age      = 22
2  citizen  = True
3  has_id   = True
4  is_minor = False
5
6  # AND -- all must be True
7  if age >= 18 and citizen and has_id:
8      print("Can register to vote.")
9
10 # OR -- at least one must be True
11 if age < 18 or not citizen:
12     print("Cannot vote.")
13 else:
14     print("Voting conditions met.")
15
16 # NOT -- inverts the Boolean
17 if not is_minor:
18     print("User is an adult.")
```

Listing 15: AND, OR, NOT in if conditions

```
Can register to vote.
Voting conditions met.
User is an adult.
```

## 6.7 Pattern 7 — Comparison Operators Reference

| Op                 | Meaning          | Example (x=5)                  |
|--------------------|------------------|--------------------------------|
| <code>==</code>    | Equal to         | <code>x == 5</code> → True     |
| <code>!=</code>    | Not equal to     | <code>x != 3</code> → True     |
| <code>&gt;</code>  | Greater than     | <code>x &gt; 3</code> → True   |
| <code>&lt;</code>  | Less than        | <code>x &lt; 10</code> → True  |
| <code>&gt;=</code> | Greater or equal | <code>x &gt;= 5</code> → True  |
| <code>&lt;=</code> | Less or equal    | <code>x &lt;= 4</code> → False |

## 6.8 Pattern 8 — Chained Comparisons

Python allows **chained comparisons** — an elegant Pythonic feature not found in most languages.

```

1 x = 15
2
3 # Traditional approach (two conditions joined by and)
4 if x > 10 and x < 20:
5     print("x is between 10 and 20")
6
7 # Pythonic chained comparison (equivalent, cleaner)
8 if 10 < x < 20:
9     print("x is between 10 and 20 (chained)")
10
11 mark = 65
12 if 50 <= mark <= 69:
13     print("Grade: C")

```

Listing 16: Chained comparison — Pythonic style

```

x is between 10 and 20
x is between 10 and 20 (chained)
Grade: C

```

## 6.9 Pattern 9 — Membership Tests with `in` and `not in`

```

1 fruits      = ["apple", "banana", "cherry"]
2 user_input  = "banana"
3
4 if user_input in fruits:
5     print(f"{user_input} is in the list!")
6
7 vowels = "aeiou"
8 letter = "e"
9
10 if letter in vowels:
11     print(f"'{letter}' is a vowel.")
12
13 if "grape" not in fruits:
14     print("Grape is not in the list.")

```

Listing 17: in / not in inside conditions

```
banana is in the list!
'e' is a vowel.
Grape is not in the list.
```

### 6.10 Pattern 10 — Identity Tests with is and is not

`is` checks **object identity** (same memory address), not value equality.

```
1 x = None
2
3 if x is None:
4     print("x has no value assigned.")    # runs
5
6 a = [1, 2, 3]
7 b = a                                     # b points to the SAME list object
8 c = [1, 2, 3]                             # c is a DIFFERENT object with equal content
9
10 if a is b:
11     print("a and b are the same object.")    # runs
12 if a is not c:
13     print("a and c are different objects.")    # runs despite equal
                                                values
```

Listing 18: is / is not – identity checks

```
x has no value assigned.
a and b are the same object.
a and c are different objects.
```

### 6.11 Pattern 11 — Truthy and Falsy Values

Python evaluates many types as **True** or **False** without an explicit comparison.

| Falsy (evaluates as False) | Truthy (evaluates as True)        |
|----------------------------|-----------------------------------|
| False                      | True                              |
| 0 (integer zero)           | Any non-zero number               |
| 0.0 (float zero)           | Any non-empty string              |
| "" (empty string)          | Any non-empty list/tuple/dict/set |
| [] (empty list)            | Any object not listed as Falsy    |
| { } (empty dict/set)       |                                   |
| None                       |                                   |

```
1 name = ""
2 score = 0
3 items = [1, 2, 3]
4
5 if name:
6     print("Name is set.")
```

```

7  else:
8      print("Name is empty!")          # "" is falsy -- runs
9
10 if score:
11     print("Has score.")
12 else:
13     print("Score is zero!")          # 0 is falsy -- runs
14
15 if items:
16     print(f"List has {len(items)} items.") # non-empty list is
    truthful

```

Listing 19: Truthy/Falsy in if conditions

```

Name is empty!
Score is zero!
List has 3 items.

```

## 6.12 Pattern 12 — Ternary (Conditional Expression)

Python's **ternary operator** expresses a simple **if-else** on a **single line**.

```
value = true_value if condition else false_value
```

```

1  age      = 20
2  status = "Adult" if age >= 18 else "Minor"
3  print(status)      # Adult
4
5  x        = 10
6  label = "Positive" if x > 0 else ("Zero" if x == 0 else "Negative"
7  )
8  print(label)      # Positive
9
10 # Nested ternary for grading (use sparingly -- hurts readability)
11 score = 75
12 result = "A" if score >= 90 else "B" if score >= 80 else "C" if
    score >= 70 else "F"
13 print(result)      # C

```

Listing 20: Ternary conditional expression

```

Adult
Positive
C

```

## 6.13 Pattern 13 — if Calling Functions

Conditions can call **functions** that return Boolean values.

```

1  def is_prime(n):
2      if n < 2:
3          return False
4      for i in range(2, int(n**0.5) + 1):
5          if n % i == 0:
6              return False

```



```

7     return True
8
9 number = 17
10 if is_prime(number):
11     print(f"{number} is a Prime number.")
12 else:
13     print(f"{number} is NOT a Prime number.")

```

Listing 21: if with function calls

```
17 is a Prime number.
```

#### 6.14 Pattern 14 — if Inside Loops (FizzBuzz)

```

1 for i in range(1, 21):
2     if i % 15 == 0:           # divisible by both 3 and 5
3         print("FizzBuzz")
4     elif i % 3 == 0:
5         print("Fizz")
6     elif i % 5 == 0:
7         print("Buzz")
8     else:
9         print(i)

```

Listing 22: if-elif-else inside a for loop – FizzBuzz

```

1  2  Fizz  4  Buzz  Fizz  7  8  Fizz  Buzz  11  Fizz  13  14
   FizzBuzz 16  17  Fizz  19  Buzz

```

#### 6.15 Pattern 15 — if with User Input

```

1 num1 = float(input("Enter first number: "))
2 op   = input("Enter operator (+, -, *, /): ")
3 num2 = float(input("Enter second number: "))
4
5 if op == "+":
6     result = num1 + num2
7 elif op == "-":
8     result = num1 - num2
9 elif op == "*":
10    result = num1 * num2
11 elif op == "/":
12    if num2 != 0:           # nested check to prevent division by
13        result = num1 / num2    zero
14    else:
15        result = None
16        print("Error: Division by zero!")
17 else:
18    result = None
19    print("Unknown operator!")
20
21 if result is not None:
22    print(f"Result: {num1} {op} {num2} = {result}")

```

Listing 23: if-else with user input: simple calculator

## 6.16 Pattern 16 — if in List / Dict Comprehensions

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2
3 # Keep only even numbers (filter)
4 evens = [n for n in numbers if n % 2 == 0]
5 print("Evens:", evens)
6
7 # Label each number (transform with if-else)
8 labels = ["Even" if n % 2 == 0 else "Odd" for n in numbers]
9 print("Labels:", labels)
10
11 # Dict comprehension with condition
12 squares = {n: n**2 for n in numbers if n > 5}
13 print("Squares > 5:", squares)
```

Listing 24: Conditional in list and dict comprehensions

```
Evens: [2, 4, 6, 8, 10]
Labels: ['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even']
Squares > 5: {6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

## 7 Real-World Application Examples

### 7.1 ATM Simulation

```
1 pin_correct = True
2 balance     = 1500
3 withdraw    = 2000
4
5 if pin_correct:
6     print("PIN accepted.")
7     if withdraw > 0:
8         if withdraw <= balance:
9             if withdraw % 100 == 0:      # ATMs dispense multiples
10                of 100
11                balance -= withdraw
12                print(f"Dispensing ${withdraw}.")
13                print(f"Remaining balance: ${balance}")
14            else:
15                print("Amount must be a multiple of 100.")
16        else:
17            print("Insufficient funds.")    # 2000 > 1500 -- runs
18            here
19    else:
20        print("Invalid withdrawal amount.")
21else:
22    print("Incorrect PIN.")
```

Listing 25: ATM System with nested if-else

```
PIN accepted.  
Insufficient funds.
```

## 7.2 Traffic Light System

```
1 light      = "red"  
2 emergency = False  
3  
4 if emergency:  
5     print("Emergency vehicle -- all clear!")  
6 else:  
7     if light == "green":  
8         print("Go!")  
9     elif light == "yellow":  
10        print("Slow down -- prepare to stop.")  
11    elif light == "red":  
12        print("Stop!")  
13    else:  
14        print("Malfunctioning signal -- proceed with caution.")
```

Listing 26: Traffic light decision system

```
Stop!
```

## 7.3 E-Commerce Discount Engine

```
1 cart_total    = 1200  
2 is_member     = True  
3 coupon_valid  = True  
4 coupon_code   = "SAVE20"  
5  
6 discount = 0  
7  
8 if is_member:  
9     discount += 10           # 10% member discount  
10    if cart_total > 1000:  
11        discount += 5       # extra 5% for large orders  
12    if coupon_valid:  
13        if coupon_code == "SAVE20":  
14            discount += 20   # 20% coupon  
15        else:  
16            discount += 5    # generic 5% coupon  
17 else:  
18     if cart_total > 2000:  
19         discount += 5       # 5% for non-members on big orders  
20  
21 final_price = cart_total * (1 - discount / 100)  
22 print(f"Original:  ${cart_total}")  
23 print(f"Discount:  {discount}%")  
24 print(f"You pay:   ${final_price:.2f}")
```

Listing 27: Discount engine with nested conditions

```
Original:   $1200
Discount:   35%
You pay:    $780.00
```

## 7.4 Weather Advisory System

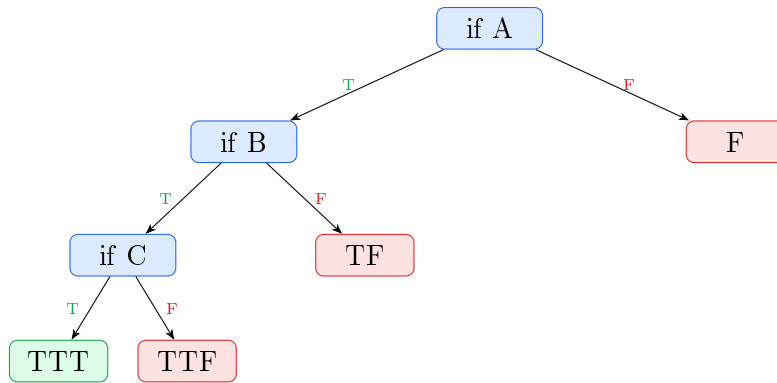
```
1 temp      = 38      # Celsius
2 humidity  = 90      # percent
3 wind_kmh  = 15
4
5 print("=== Weather Advisory ===")
6
7 if temp > 40:
8     if humidity > 85:
9         print("EXTREME DANGER: Heat index critical!")
10    else:
11        print("DANGER: Extreme heat. Stay indoors.")
12 elif temp > 35:
13     if humidity > 80:
14         print("WARNING: High temp + humidity. Heat stroke risk.")
15     else:
16         print("CAUTION: Hot day. Drink plenty of water.")
17 elif temp > 25:
18     print("Pleasant weather.")
19 else:
20     if wind_kmh > 50:
21         print("Cold and windy. Dress warmly.")
22     else:
23         print("Cool weather. Light jacket recommended.")
24
25 if wind_kmh > 60:
26     print("ALERT: Strong winds!")
```

Listing 28: Multi-factor weather advisory

```
=== Weather Advisory ===
WARNING: High temp + humidity. Heat stroke risk.
```

## 8 Nesting Depth — Visual Decision Tree

The tree below shows how a **3-level nested** if-else creates up to 4 possible outcomes. Each **T** branch means the condition was **True**; **F** means **False**. Block **TTT** is reached *only* when all three conditions are **True** simultaneously.



## 9 The pass Statement in Conditionals

`pass` is a **null operation** — it does nothing but satisfies Python's requirement for a non-empty block. Used as a placeholder during development.

```

1 x = 10
2
3 if x > 5:
4     pass          # TODO: implement this later
5 else:
6     print("x is 5 or less")
7
8 # Common use: stub functions
9 def handle_error(code):
10     if code == 404:
11         pass      # TODO: implement 404 handler
12     elif code == 500:
13         print("Server error!")

```

Listing 29: `pass` as a placeholder in if blocks

## 10 Guard Clauses — Refactoring Deep Nesting

**Guard clauses** (early returns) are the professional way to eliminate deep nesting in functions.

```

1 # DEEPLY NESTED (hard to read -- 6 levels deep)
2 def process_order(user, item, quantity):
3     if user is not None:
4         if user.is_active:
5             if item is not None:
6                 if quantity > 0:
7                     if quantity <= item.stock:
8                         return "Order placed!"
9                     else:
10                        return "Out of stock"
11                else:
12                    return "Invalid quantity"
13            else:
14                return "Item not found"
15        else:
16            return "Account inactive"

```

```
17     else:
18         return "User not found"
19
20
21 # GUARD CLAUSES (clean, flat, readable)
22 def process_order_clean(user, item, quantity):
23     if user is None:
24         return "User not found"
25     if not user.is_active:
26         return "Account inactive"
27     if item is None:
28         return "Item not found"
29     if quantity <= 0:
30         return "Invalid quantity"
31     if quantity > item.stock:
32         return "Out of stock"
33     return "Order placed!"    # happy path stays unindented at the
                               end
```

Listing 30: Deep nesting vs guard clauses

### Guard Clause Principle

*“Handle error and edge cases first, so the happy path stays unindented and readable.”*  
This is also known as the **early return** or **fail-fast** pattern. It dramatically reduces nesting and improves maintainability.

## 11 Common Mistakes and How to Avoid Them

| Mistake                | Wrong                                               | Correct                                               |
|------------------------|-----------------------------------------------------|-------------------------------------------------------|
| Missing colon          | <code>if x &gt; 5</code>                            | <code>if x &gt; 5:</code>                             |
| Using = instead of ==  | <code>if x = 5:</code>                              | <code>if x == 5:</code>                               |
| Wrong indentation      | Body at same level as <code>if</code>               | Body indented 4 spaces                                |
| Dangling else          | <code>else</code> attached to wrong <code>if</code> | Match <code>else</code> indent to its <code>if</code> |
| Comparing None with == | <code>if x == None:</code>                          | <code>if x is None:</code>                            |
| elif after else        | <code>else: ... elif ...:</code>                    | <code>else</code> must be last                        |
| Mixing tabs & spaces   | Inconsistent indent                                 | Use only 4 spaces                                     |
| Missing colon on else  | <code>else</code>                                   | <code>else:</code>                                    |

## 12 All Variants — Summary Overview

| Construct                      | Use When                   | Blocks Run      |
|--------------------------------|----------------------------|-----------------|
| <code>if cond:</code>          | Optional action            | 0 or 1          |
| <code>if...else</code>         | Binary choice              | Exactly 1       |
| <code>if...elif...else</code>  | Multiple exclusive choices | Exactly 1       |
| Nested if-else                 | Hierarchical conditions    | Exactly 1 path  |
| Multiple if                    | All conditions independent | 0 to N          |
| Ternary                        | Simple single-line assign  | Exactly 1 value |
| <code>if x is None</code>      | Absence-of-value check     | 0 or 1          |
| <code>if x:</code>             | Truthy shorthand           | 0 or 1          |
| <code>if a and b:</code>       | Both must hold             | 0 or 1          |
| <code>if a or b:</code>        | Either may hold            | 0 or 1          |
| <code>if not a:</code>         | Negation check             | 0 or 1          |
| <code>if x in seq:</code>      | Membership test            | 0 or 1          |
| <code>10 &lt; x &lt; 20</code> | Range / chained comparison | 0 or 1          |

## 13 Best Practices Summary

1. **Use 4 spaces** per indentation level (PEP 8). Never mix tabs and spaces.
2. **Prefer `elif` over `nested if`** when conditions are mutually exclusive.
3. **Use guard clauses** (early return) to keep nesting shallow in functions.
4. **Use `is`/`is not`** for `None` checks, not `==`/`!=`.
5. **Leverage `truthy/falsy`**: `if items:` beats `if len(items) > 0:`.
6. **Use chained comparisons**: `10 < x < 20` is cleaner than two conditions.
7. **Avoid nesting beyond 3 levels** — refactor into helper functions.
8. **Never use `=` inside a condition** — use `==` for equality testing.
9. **Name Boolean variables clearly**: `is_active`, `has_permission`.
10. **Add comments** to complex nested blocks explaining *why*, not just *what*.
11. **Test all branches** of nested if-else — every path must be verified.
12. **Use `pass`** as a temporary placeholder while developing stub code.

## 14 Conclusion

Python's `if-elif-else` system is the **backbone of decision-making** in every Python program. Here is a summary of everything covered:



- **if** — one condition, optional single branch action.
- **if-else** — binary choice; exactly one of two paths is taken.
- **if-elif-else** — multi-way branch; the first **True** condition wins.
- **Nested if-else** — hierarchical decisions; inner conditions only evaluated if outer is **True**.
- **Ternary expression** — compact single-line conditional value assignment.
- **Logical operators** (**and**, **or**, **not**) — combine multiple conditions.
- **Membership tests** (**in**, **not in**) — check presence in sequences.
- **Identity tests** (**is**, **is not**) — compare object identity (use for **None**).
- **Truthy/Falsy** — Python implicitly converts many types to Boolean.
- **Chained comparisons** — Pythonic range checks like `0 <= x < 100`.
- **Guard clauses** — eliminate deep nesting using early returns.
- **Indentation is syntax** — it defines block ownership and nesting.

Mastering these patterns lets you model any real-world logic — from simple checks to complex multi-factor decision engines — in clean, readable, and maintainable Python code.

— End of Document —